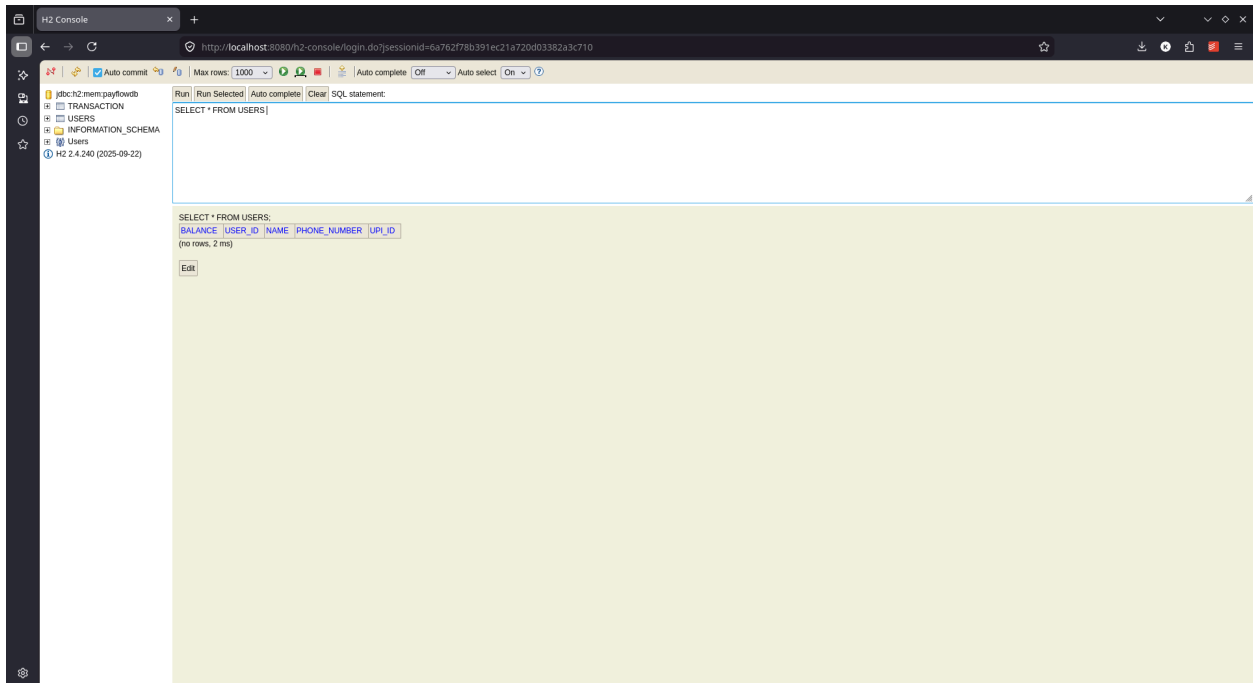
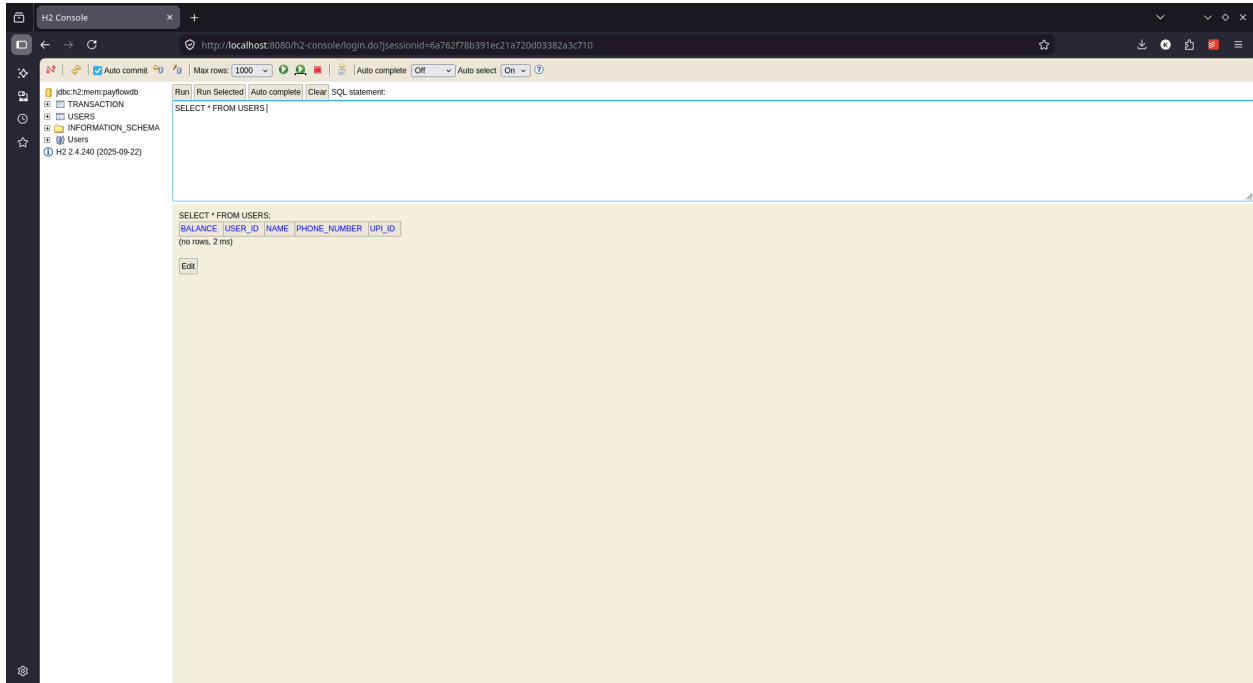


Task 2 Screenshots

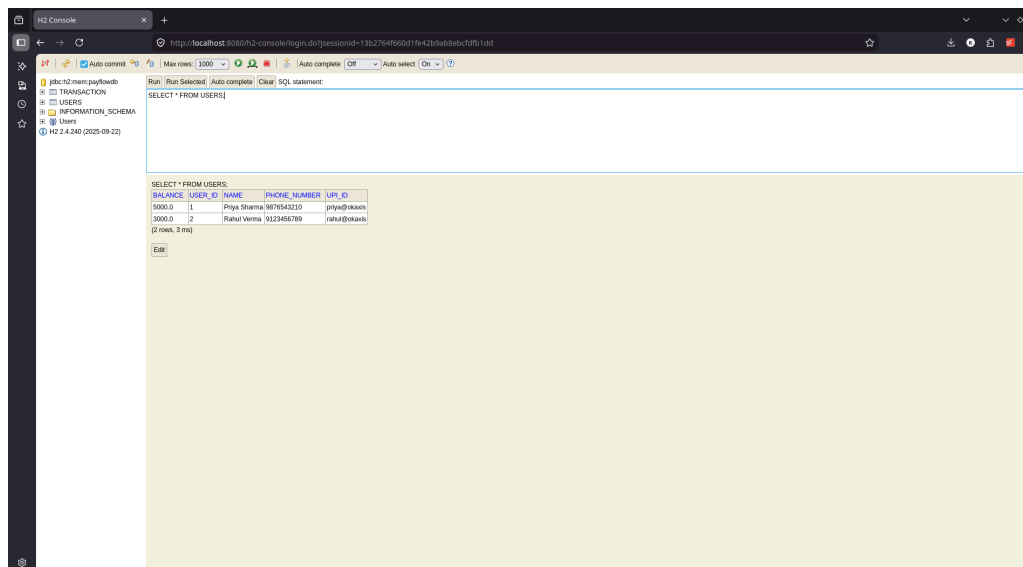
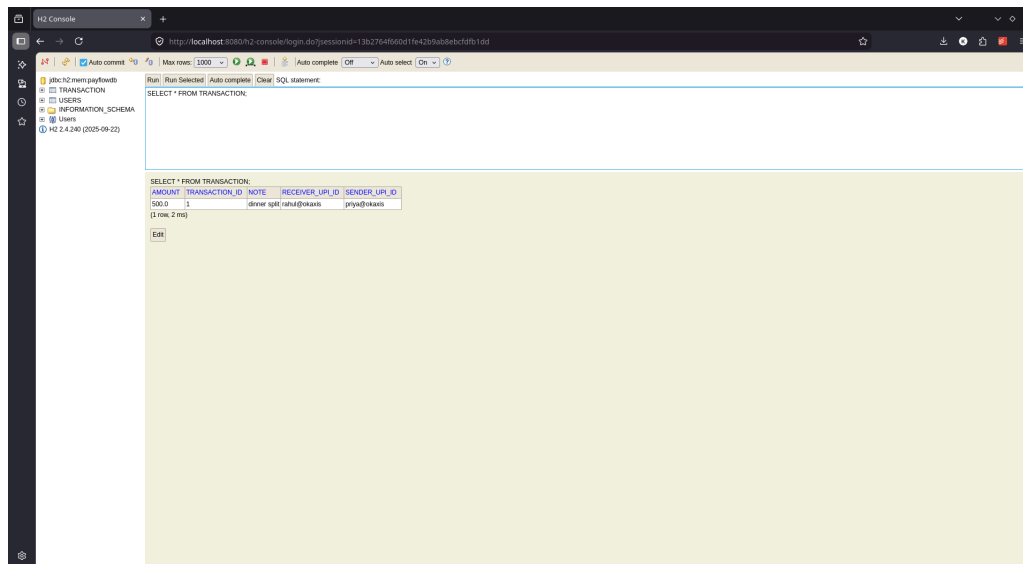


Task 5

Answer

Without `@RequestBody`, when I called POST `/users` with Priya's details, Spring completely ignored the JSON body. The `registerUser` method received an empty `User` object where `name`, `upild`, `balance` and `phoneNumber` were all null — because Spring didn't know it needed to parse the request body. Adding `@RequestBody` tells Spring to read the incoming JSON and map it into the Java `User` object automatically using Jackson.

Screenshots



Conceptual Write Up

Question 1 — Request Lifecycle

When I send a `curl POST /users` request, it first hits Spring's `DispatcherServlet`, which acts like a front desk that receives all incoming HTTP traffic. The `DispatcherServlet` figures out which controller should handle this request and passes it to a `RequestMappingHandlerAdapter`. This adapter then calls my `UserController.createUser` method, passing in the JSON body so the user can be registered and saved to the database.

Question 2 — Serialisation

A built-in library called **Jackson** automatically converts the incoming JSON `{"name":"Priya Sharma","upiId":"priya@okaxis"}` into a Java User object. Jackson does this by matching JSON keys exactly to Java field names. If the key was `"upi_id"` instead of `"upiId"`, Jackson wouldn't find a matching field by default and that field would be left as `null` in the Java object — this is because Jackson follows camelCase by default, not snake_case.

Question 3 — Spring Boot Features

Spring Boot has three main features. First, the **Embedded Server** — Tomcat started automatically on port 8080 when I ran PayFlow, with no manual server setup needed. Second, **Auto-configuration** — Spring Boot detected H2 and JPA in my `pom.xml` and automatically set up the database, the H2 console, and the `DataSource` without any XML. Third, **Production-ready Defaults** — HikariCP connection pooling was set up automatically the moment the app started, which I could see in the startup logs as `HikariPool-1 - Start completed`.

Question 4 — Spring vs Spring Boot

With plain Spring, I would have had to manually write XML configuration files to set up the database connection, configure Hibernate, and deploy the app to a separately installed Tomcat server. Spring Boot handles all of this automatically using sensible defaults and auto-configuration. In PayFlow specifically, I never wrote a single line of XML — the embedded

server, H2 setup, and JPA configuration all happened automatically just by adding the right dependencies.

Question 5 — Stateless REST

Stateless means the server does not remember anything about previous requests — every request must contain all the information needed to process it on its own. This matters a lot if PayFlow runs on three servers behind a load balancer. If Priya's first request goes to Server 1 and her next request goes to Server 2, Server 2 can still handle it perfectly because it doesn't rely on any memory stored only on Server 1 — each request is completely self-contained.

Question 6 — Persistence

If transaction records were stored in a Java List, all data would be completely wiped the moment the server restarts — because a List only lives in memory while the program is running. I actually saw this firsthand with H2 — since it is an in-memory database, all my test data disappeared every time I restarted the app. For a real payments app this is completely unacceptable because financial records must be permanently saved, which is why production apps use persistent databases like PostgreSQL or MySQL instead.

@RequestBody Explanation

The `@RequestBody` annotation tells Spring to read the incoming HTTP request body and convert the JSON into a Java object. Without it, when I called `POST /users` with Priya's details, Spring would completely ignore the JSON body. The `registerUser` method would receive an empty User object where name, upId, balance, and phoneNumber are all `null` — because Spring wouldn't know it needed to parse the request body at all.
